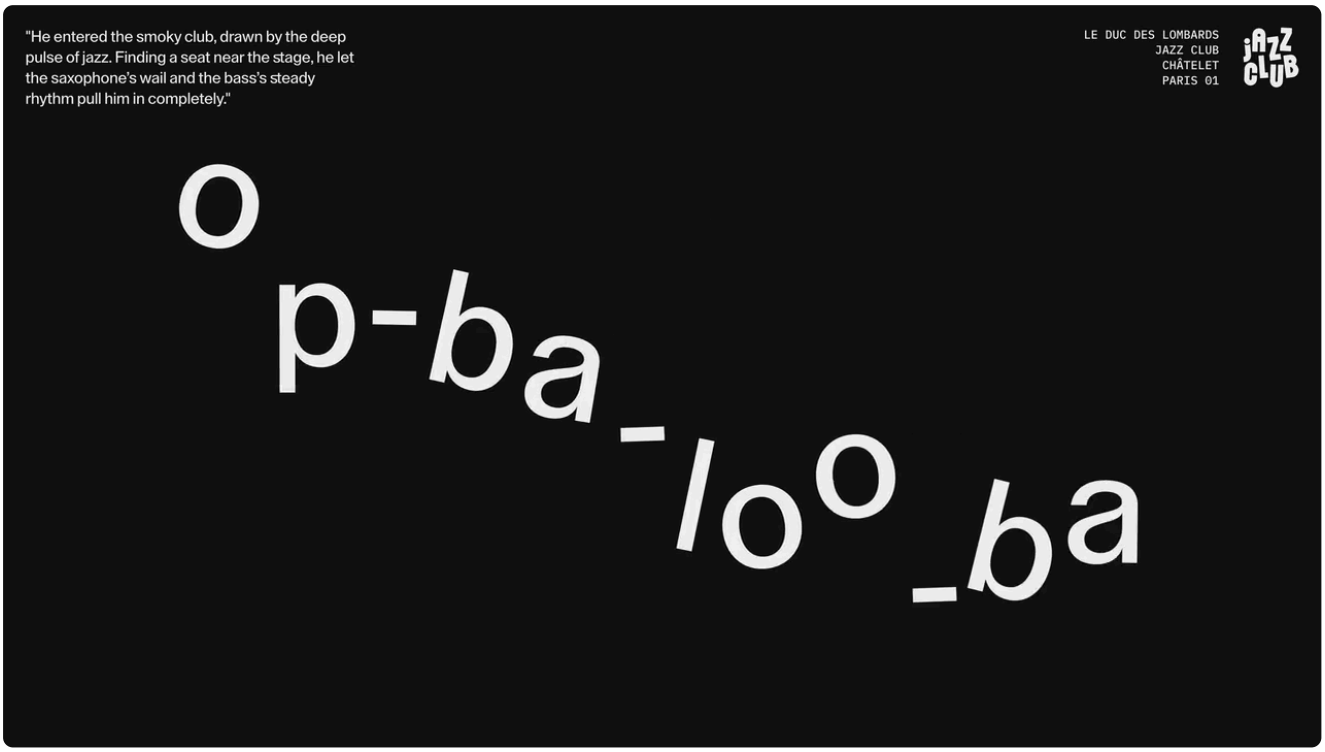
Made With Gsap

010
Jazzy letters



In this effect, we'll make a sentence scroll from right to left as the user scrolls. The letters will appear and disappear dynamically based on their position at the edges of the viewport.
To make it more visually interesting, we'll arrange the letters along a sine wave that we can tweak. Doesn't the result have a bit of a jazz vibe? Then let's go—time to practice our scat singing skills!

HTML Structure

The structure is relatively simple. The sentence inside our `container` will move from right to left while remaining visible within the viewport.

```
<section class="mwg_effect010">
    <div class="container">
        <p class="text">Shoo-ba-lah-dee, bop-ba-loo-bah. Skee-bop-a-dee!</p>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Some CSS

To prevent a horizontal scrollbar when the phrase extends beyond the viewport, we hide any overflow from the root element of our component:

```
.mwg_effect010{
    overflow: hidden;
}
```

The properties `white-space: nowrap` and `width: max-content` ensure that the sentence does not wrap and maintains its full width. This will be crucial for our upcoming calculations.

```
.mwg_effect010 .text {
    padding: 0 100vw; // To avoid seeing the sentence directly
    display: flex;
    width: max-content;
    white-space: nowrap;
}
```

To allow each letter of our sentence to undergo transformations, we set them to `display: inline-block`. By default, letters are hidden and slightly spaced out for better readability.

```
.mwg_effect010 .letter {
    display: inline-block;
    visibility: hidden;
    padding: 0 0.2vw;
}
```

Font loading

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

correctly and that the wait for the font to be fully loaded before proceeding.

```
document.fonts.ready.then(() => {
    // Once the font is loaded, let the fun begin!
})
```

Once the font is loaded, let the fun begin! I created a utility function called `wrapLettersInSpan()` to separate each letter and wrap it in a `<span>`. This function takes as a parameter the DOM element containing the phrase we want to split.

```
const text = document.querySelector('.mwg_effect010 .text')
wrapLettersInSpan(text)

function wrapLettersInSpan(element) {
    const text = element.textContent;
    element.innerHTML = text
        .split('')
        .map(char => char === ' ' ? '<span> </span>' : `<span class="letter">${char}</s
        .join('');
}
```

To achieve this, I use the `split()` method to break the text into an array, using each character (`''`) as a separator. The `map()` function then wraps each character in a `<span>`. One special case: if the current character is a space, we replace it with ` `, which represents a non-breaking space.

Finally, I reconstruct the phrase by placing each character back in order. The newly modified string (with its DOM) is then returned to the original element in parameter.

Moving the Sentence

Now that our sentence is made up of elements, we can move it from right to left. I start by calculating the scroll distance:

```
const distance = text.clientWidth - document.body.clientWidth
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

creen, which is why I subtract

Made With Gsap

at the translation stops exactly when the end of the sentence reaches the right edge of the viewport. Why not use `window.innerWidth` to calculate the screen width? Because `document.body.clientWidth` takes into account the browser's scrollbar width.

I use the `gsap.to()` method to create the tween animation. Inside its `scrollTrigger` object, I specify that I want to `pin` the `container` during the animation. The `scrollTrigger` object also allows me to define when the tween animation should trigger.

```
const scrollTween = gsap.to(text, {
    x: - distance, // Translation on the x-axis
    ease: 'none', // Linear movement
    scrollTrigger: {
        trigger: '.mwg_effect010 .container', // Listen to the container's position
        pin: true, // We pin the container division
        end: '+=' + distance // We unpin it at the end of the animation
        scrub: true, // Progress with scrolling
    }
});
```

Notice that the tween is stored in the variable `scrollTween`, which we will use in the next step.
To go further, we could choose a different scrolling speed in the `scrollTween` variable, for example `end: '+=' + distance / 2` to make the text scroll twice as fast compared to the default scroll speed.

Positioning the letters

To apply different values to each letter, I use JavaScript's native `forEach()` function. This allows us to track the position of each letter individually inside our `scrollTrigger` objects.

```
let sinusIncr = 0
letters.forEach( (letter, index) => {
    ...
})
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

start with the native function between -1 and 1 based on the input parameter. If this parameter is continuously incremented, the function will progressively return values from -1 to 1, then back from 1 to -1, and so on. To increase the amplitude, we simply multiply the returned value by a given factor.

Inside the `forEach()` loop, the variable `sinusIncr` is incremented at each iteration. The corresponding letter is then positioned along the sinusoidal curve. The amplitude is calculated using `window.innerHeight` to keep the effect responsive based on the viewport height:

```
y: Math.sin(sinusIncr) * (window.innerHeight / 5.6),
```

To modify the frequency of the sinusoid, we just need to adjust the increment value at each iteration.

```
sinusIncr += 0.3
```

To correctly position each letter, I use `gsap.set()`, which immediately applies the properties to the target element.

```
letters.forEach(letter => {
    gsap.set(letter, {
        // Set the y position along the sinusoid
        y: Math.sin(sinusIncr) * (window.innerHeight * 0.2),
        autoAlpha:1, // Make the letter appear
        immediateRender:false, // To not erase the initial state
        scrollTrigger:{
            trigger:letter, // Listen to the letter’s position
            containerAnimation: scrollTween, // Contains the translation tween
            toggleActions: "play reverse play reverse",
            // Letter appears when its right edge is at 90% of the screen
            start: 'right 90%',
            // Letter disappears when its left edge is at 10% of the screen
            end: 'left 10%',
        }
    })
    sinusIncr += 0.3;
})
```

BATCH 001 — 050 EFFECTS

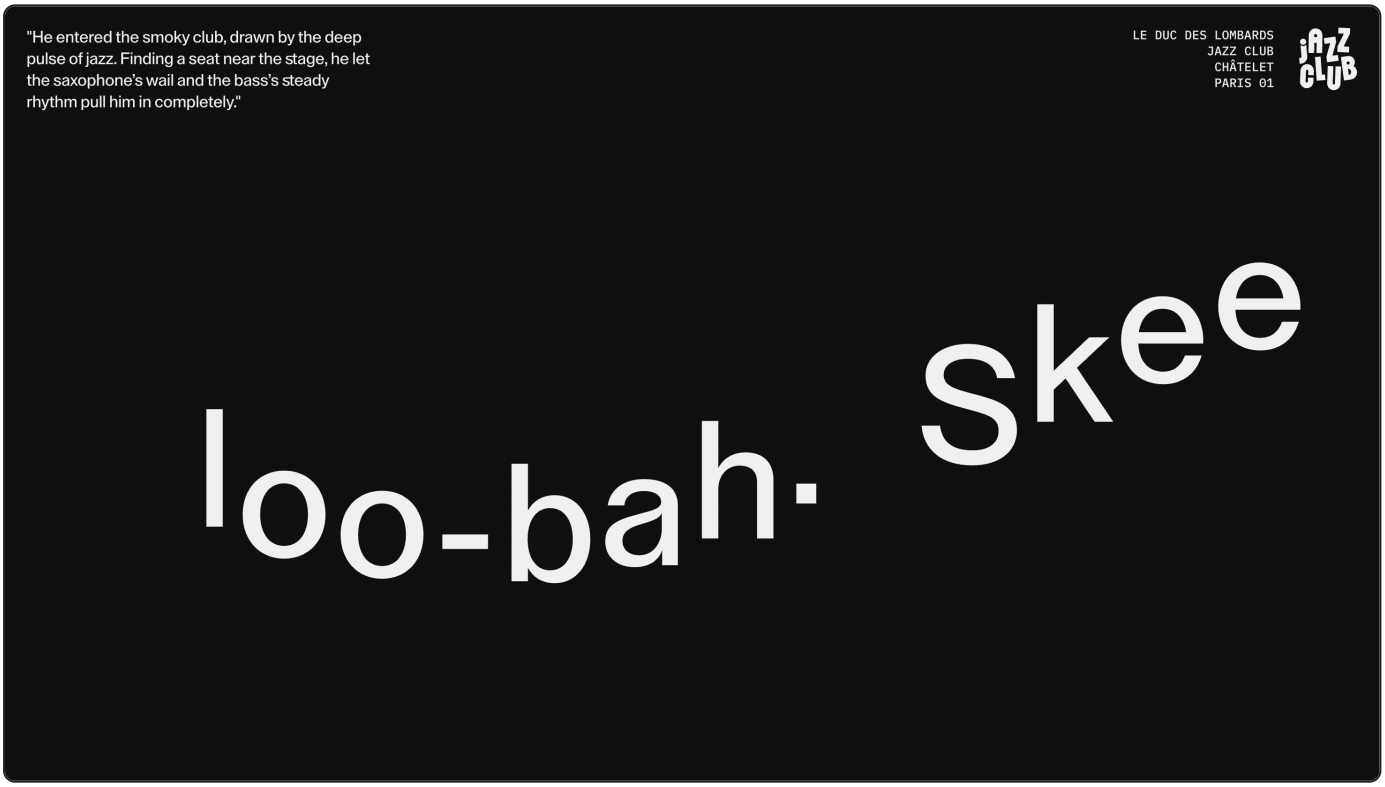
COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

e earlier. Now, we specify that linked to the container's movement by setting `containerAnimation: scrollTween`. This method is recommended for animating child elements within a horizontal scrolling effect, as it allows us to use `left` or `right` values in the `start` and `end` properties.

In the `scrollTrigger` object, what does the `toggleActions` attribute do? It controls what happens during the four different scrolling states: `onEnter`, `onLeave`, `onEnterBack` et `onLeaveBack`. Here :

- `play` : The animation starts when the start point is reached (letter appears).
- `reverse` : The animation resets when it reaches the end (letter disappears).
- `play` : If the user scrolls back up after passing the end point, the animation plays again (letter appears).
- `reverse` : The animation resets when fully rewound (letter disappears).

Here is the current result:



A bit more randomness

Right now, the sinusoidal curve is too predictable. It's time to mix things up a bit!
To do this, we'll use `Math.random()`. As a reminder, `Math.random() - 0.5` returns a value between -0.5 and 0.5. When we multiply this value by a chosen number, we

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

ic to two properties:

Made With Gsap

ement on the y-axis relative to the letter's height) and the `rotation` attribute.

```
letters.forEach(letter => {
    gsap.set(letter, {
        y: Math.sin(sinusIncr) * (window.innerHeight * 0.2),
        yPercent: (Math.random() - 0.5) * 100, // Between -50 & 50
        rotation: (Math.random() - 0.5) * 32, // Between -16 & 16
        autoAlpha:1, // Make the letter appear
        immediateRender:false, // Do not erease the initial state
        scrollTrigger:{
            trigger:letter, // Listen to the letter's position
            containerAnimation: scrollTween, // Contains the translation tween
            toggleActions: "play reverse play reverse",
            // Letter appears when its right edge is at 90% of the screen
            start: 'right 90%',
            // Letter disappears when its left edge is at 10% of the screen
            end: 'left 10%',
        }
    })

    // Control the frequency of the sinusoid
    sinusIncr += 0.3;
})
```

Go further

To modify the look of the sinusoid, I encourage you to experiment with its frequency and amplitude. You can also increase the random value ranges for an even more messy result.

In this example, I used Lenis smooth scroll to further enhance the fluidity and smoothness. If you'd like to apply it as well, check out our Go Further page to learn how to add it.

Final code

HTML CSS JAVASCRIPT

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

```
    <div class="container">
        <div class="header">
            <p class="left">"He entered the smoky club, drawn by the deep pulse of jaz
            <div class="right">
                <p>Le Duc des Lombards <br/>Jazz Club<br/>Châtelet <br/>Paris 01</p>
                <img class="jazzdog" src="assets/medias/jazzclub.svg" alt="">
            </div>
        </div>
        <p class="text">Shoo-ba-lah-dee, bop-ba-loo-bah. Skee-bop-a-dee, bop-bow-wow-w
    </div>
</section>
```

Wanna learn something else?

BACK TO THE COLLECTION

Never miss what's next

Your email SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.

SOCIAL

X(TWITTER)

PAGES

HOME

BATCH 001 – 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

-

PRICING

CONTACT

REACH US

SUPPORT

Made With Gsap

2025©

PERMISSIONS & TERMS

PRIVACY POLICY